



De La Salle University

Gokongwei College of Engineering

Department of Electronics and Computer Engineering

Numerical Methods - E32

**Voltage Estimation in a Four-to-Six Load Bus Microgrid using the
Gauss-Seidel Iterative Method**

Submitted by:

Cuartero, Gavin Joaquin

Tafalla, Carl Andrei

Tonducan, Zach Nathan

I. Introduction

Microgrids are groups of interconnected loads that are increasingly used to support localized energy systems, especially in areas that have either unstable grid connections, or a supporting secondary renewable energy source. They typically consist of several interconnected loads, called buses, where electrical power is generated, consumed, and distributed across the grid (National Laboratory of the Rockies, 2025). In more modern power systems and especially with the advent of renewable energy sources, microgrids arise as a more recognizable solution that is easier to set up and operate. However, it comes with the flaw that large-scale grids often set aside; the critical sensitivity to power and voltage changes, which enables a need for a specific and accurate computation for efficiency and improvement (Center for Climate and Energy Solutions, n.d.).

To ensure the usability and efficiency of microgrids within the systems that are built, engineers are often required to analyze voltage magnitudes and phases at each bus in the system. This process is often called power flow, or load flow analysis, often done with the help of matrix and array-oriented programming languages and environments like MATLAB or Python. Load flow analyses often require a series of nonlinear equations that come from the total generated and consumed voltages at each node or bus, especially since generator buses may elicit complex voltages and phasor values. However, these equations are often simplified for instructional or computational purposes (Yang et al, 2019), as within concerns of voltage calculations, voltage magnitudes and angles are solved as dependent variables based on specified power injections, which is usually taken in these types of analyzed systems.

In the context of microgrids, reliable power flow solutions are essential for ensuring voltage stability, minimizing losses, preventing equipment damage, and enabling effective energy management. Without accurate bus voltage profiles, operators cannot make informed decisions about reactive power compensation or protection coordination for the microgrids' proper and safe production of energy. Within houses and residential areas in the Philippines, where microgrids appear more often due to a prevalence of smaller-scale energy generation and production systems likely due to logistical problems that make centralized grid systems more unfeasible, this analysis proves to be more critical, as maximizing efficiency and safety is as important within relatively smaller communities that typically receive less attention (Chiu, 2026).

Furthermore, the prevalence of microgrids in the Philippines is exacerbated by governmental promotion and laws, like the Republic Act no. 11646 or the Microgrid Systems Act enacted in 2022, which promotes national electrification through low-cost

sources of energy that may be renewable or on a smaller scale than regional electric grids and the like. This law is reinforced through actual implementations like in Quezon, where 200 households benefit from a hybrid electricity microgrid, from both a photovoltaic solar source and a diesel-based generator (DOE, 2026).

In discussing and calculating power flow problems that often span multiple variables and equations, engineering and mathematical methods are often employed, be it analytical or numerical estimations. Among these methods, the Gauss-Seidel iterative method stands as one of the most fundamental approaches for solving systems of nonlinear equations associated with power flow analysis. Although it is often considered less efficient than more advanced methods such as Newton-Raphson in large-scale power systems (Critical Power Systems, 2025), the Gauss-Seidel method offers advantages in terms of simplicity, ease of implementation, and lower computational requirements (Mbasso et al., 2025).

These advantages and initial constraints make the method particularly suitable for smaller-scale systems such as microgrids, where the number of buses is intentionally made to be limited, and where computational resources may be constrained or more inaccessible. In such situations and environments, the relatively slower convergence of the Gauss-Seidel method can be paid attention to less, while its straightforward iterative structure allows for clear step-by-step evaluation of voltage updates at each bus. This makes it not only practical for implementation but also valuable for educational and analytical purposes.

In line with this, the study focuses on applying the Gauss-Seidel iterative method to estimate bus voltages in a simplified four-load bus microgrid, as an example, but with a capability to expand to a six-load microgrid system. By demonstrating its application within a controlled and small-scale system, the research aims to highlight the continued relevance of classical load flow methods and their effectiveness in addressing modern microgrid analysis challenges. In addition, a specific emphasis on the four-bus system allows the Gauss-Seidel method to be realized effectively as a real-world engineering tool for evaluating true problems rather than being constrained to theoretical and abstract matrices and classic systems of linear equations.

II. Objectives

This section details the developmental goals and purposes of the paper that it accomplishes within the project, structuring the framework and limitations that are purposefully set within the paper.

2.1 General Objective

- To analyze and estimate bus voltages in a four to six load bus microgrid using the Gauss–Seidel iterative method.

2.2 Specific Objectives

- To model a simplified four-load bus microgrid system suitable for load flow analysis as an example.
- To allow user input to model a microgrid system with voltages solvable using the Gauss-Seidel iterative method.
- To apply the Gauss-Seidel iterative method in solving for bus voltage magnitudes and phase angles.
- To create a working program using Python which enables the Gauss-Seidel method to be used for voltage estimation associated with the microgrid buses.
- To evaluate the convergence behavior and computational performance of the Gauss-Seidel method in a small-scale system.
- To interpret the resulting voltage profiles and assess their implications for microgrid stability and operation.
- To realize the ability of the Gauss-Seidel method as a practical and suitable method for small-scale systems that require less computational complexity.

III. Methodology

This section of the paper focuses on the implementation of the Gauss-Seidel Method on a scalable microgrid system (supporting 4 to 6 buses) using Python/Jupyter Notebook (.ipynb) as the programming software and environment for the computations. Specifically, the following progression is employed as represented below:

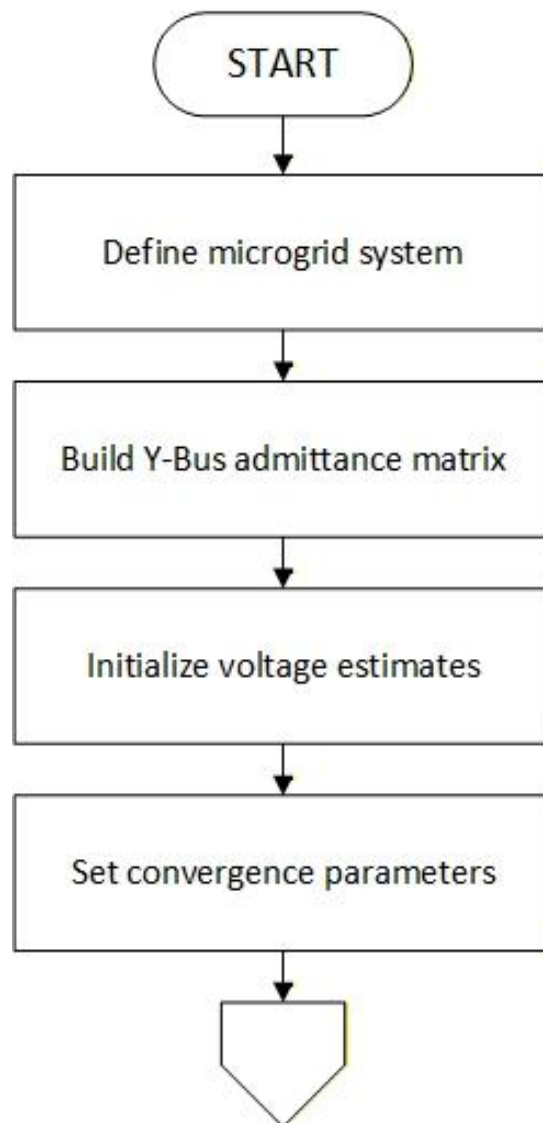


Figure 1(a). Setup flow for Gauss-Seidel Implementation.

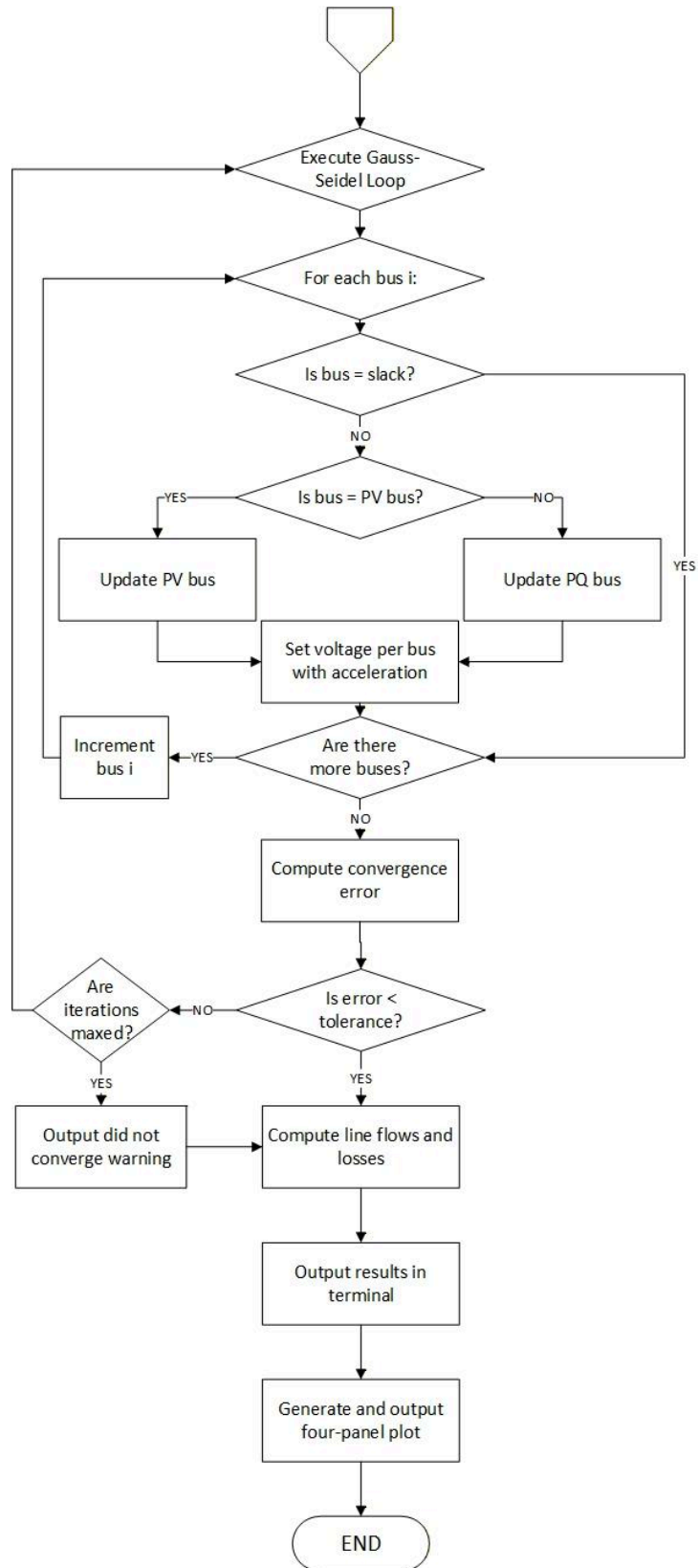


Figure 1(b). Gauss-Seidel iteration flow and output flow

3.1 *Microgrid System Description*

The microgrid system modeled in this project, which serves as the primary baseline example for the program, consists of four interconnected buses, each representing a distinct node inside a simplified residential load network. This 4-bus baseline is initialized to represent a realistic system consisting of a slack bus, a generator/PV bus, and two load/PQ buses, directly referencing Sanitha Michail et al. (2024)'s microgrid system for an islanded computation. Furthermore, the system is modeled in per-unit (pu), a normalization convention commonly used in power systems analysis to simplify calculations by eliminating the need for explicit voltage level conversions across the network.

The assignment of slack buses is mandatory in load flow analyses, where every power system must contain exactly one slack bus to serve as the voltage and phase reference. As such, Bus 1 is designated to be this slack bus with a voltage value fixed at $1.05\angle 0^\circ$ pu, which is intentionally higher than nominal values (Fiasorgbor, 2023), to reflect the typical operating conditions of slack buses that supply the rest of the network.

The second bus is assigned to be a generator/PV bus, as it represents a small distributed generator within the system that supplies supplementary power to the microgrid itself. PV buses model sources that actively regulate their terminal voltage through excitation control while injecting a scheduled amount of real power (NPTEL, n.d.). In choosing the voltage supplied by the system, a voltage setpoint of 1.02 pu and a real power injection of 0.40 pu were chosen to reflect a modest but realistic distributed generator operating within acceptable voltage limits.

For the latter buses, they are designated to be load/PQ buses, which represent residential load nodes with a voltage requirement fixed from generation patterns arbitrarily chosen to represent this project. Their scheduled power consumptions of $-0.55 - 0.18j$ pu and $-0.45 - 0.15j$ pu respectively are negative by convention, denoting net power absorption from the network. These values were selected to reflect an aggregate residential demand that the system's two supply buses: Bus 1 (Slack) and Bus 2 (PV) will have to compensate for. This entire connection thus produces a moderately loaded network suitable for demonstrating the convergence behavior of the Gauss-Seidel method without introducing numerical instability.

The five transmission lines connecting these baseline buses are characterized by their complex impedances ($R + Xj$) in per-unit, summarized in Table 1 and illustrated in Figure 2. Line impedance values were chosen to reflect typical distribution-level resistance-to-reactance ratios, where resistance is relatively significant compared to

transmission-level systems, consistent with the low-voltage characteristics of a residential microgrid. These values define the electrical coupling between buses and form the basis for constructing the nodal admittance matrix in the subsequent step.

Line	Resistance R (pu)	Reactance X (pu)	Impedance Z
Bus 1 to Bus 2	0.02	0.06	$0.02 + 0.06j$
Bus 1 to Bus 3	0.08	0.24	$0.08 + 0.24j$
Bus 2 to Bus 3	0.06	0.18	$0.06 + 0.18j$
Bus 2 to Bus 4	0.06	0.18	$0.06 + 0.18j$
Bus 3 to Bus 4	0.04	0.13	$0.04 + 0.13j$

Table 1. Complex impedance per line in 4x4 example

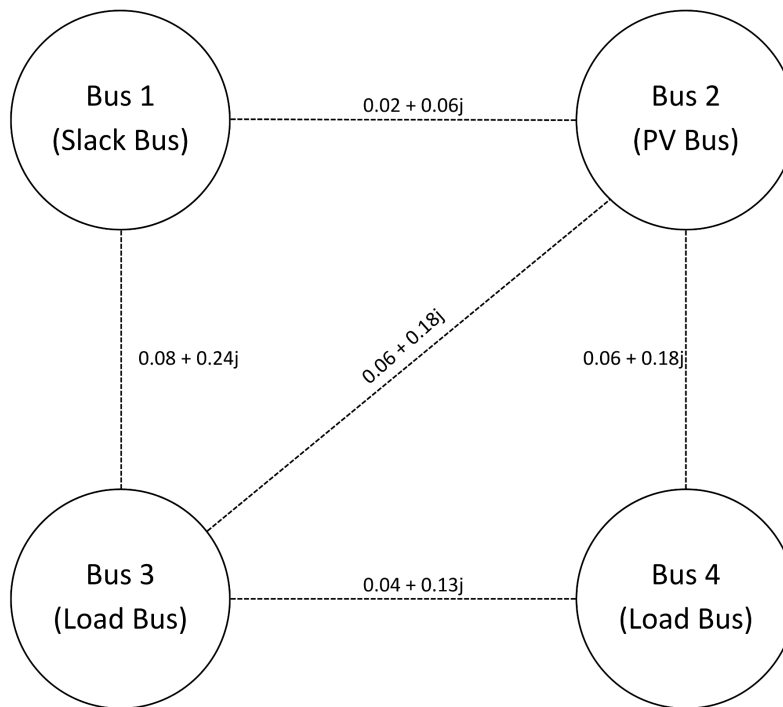


Figure 2. 4x4 example microgrid illustration with complex impedances

3.2 Nodal Admittance Matrix Formulation

Before the Gauss-Seidel algorithm can be applied to the system, the electrical relationships between all buses in the microgrid must first be expressed in a form that is suitable for computation. This is done through the utilization of the nodal admittance matrix, also known as the Y-bus matrix, which encodes admittance values of each transmission line into an n-by-n matrix that governs the relationship between bus voltages and injected currents inside the microgrid. In the case of the key baseline example, a 4-by-4 matrix is obtained, though the program dynamically scales this to an n-by-n matrix based on user configuration.

The Y-bus matrix, which is built with the `build_ybus()` function, is constructed directly from the line impedance data presented in Table 1 seen earlier. For each transmission line connecting the buses, the line admittance is first obtained by taking the reciprocal of the line impedance:

$$y_{ij} = \frac{1}{z_{ij}} = \frac{1}{R_{ij} + X_{ij}j}$$

Where each subscript i, j represents the connected buses (bus i to bus j). With that equation, the Y-bus matrix is partially populated, and their off-diagonal elements, which signify the mutual admittance between two distinct buses, are then assigned with the negative line admittance value of the line connecting them from bus i to bus j:

$$Y_{ij} = Y_{ji} = -y_{ij}$$

This construction ensures that the Y-bus matrix is symmetric, reflecting the reciprocal nature of the transmission lines, and that each diagonal entry accounts for the total admittance load seen at its corresponding bus. The resulting 4×4 Y-bus matrix for the modeled baseline microgrid is presented in Table 2. It is important to note that Bus 1 to Bus 4 and its vice-versa's admittances (Y_{14} , Y_{41}) result in a zero, considering that there is no direct transmission line connecting the slack bus into Bus 4.

$$Y_{ii} = \sum_{j \neq i} y_{ij}$$

This construction ensures that the Y-bus matrix is symmetric, reflecting the reciprocal nature of the transmission lines, and that each diagonal entry accounts for

the total admittance load seen at its corresponding bus. The resulting 4×4 Y-bus matrix for the modeled example microgrid is presented in Table 2. It is important to note that Bus 1 to Bus 4 and its vice-versa's admittances (Y_{14} , Y_{41}) result in a zero, considering that there is no direct transmission line connecting the slack bus into Bus 4.

	Bus 1	Bus 2	Bus 3	Bus 4
Bus 1	6.25 - 18.75j	-5.00 + 15.00j	-1.25 + 3.75j	0
Bus 2	-5.00 + 15.00j	8.33 - 25.00j	-1.67 + 5.00j	-1.67 + 5.00j
Bus 3	-1.25 + 3.75j	-1.67 + 5.00j	5.08 - 15.10j	-2.16 + 7.03j
Bus 4	0	-1.67 + 5.00j	-2.16 + 7.03j	3.83 - 12.03j

Table 2. 4x4 Y-Bus/nodal admittance matrix

3.3 Gauss-Seidel Implementation

The Gauss-Seidel method is applied iteratively as the `gauss_seidel_power_flow()` function to solve for the unknown bus voltages. At each iteration, every non-slack bus voltage is updated from the algorithm's computations and the most recent computed values from within the same iterations, allowing for the values' convergence to happen sooner at later computations inside the iteration rather than after the iteration themselves. The reason for the method's usage on non-slack buses is because the slack bus is a fixed value with its magnitude and phasor direction, $1.05\angle 0^\circ$ pu, which means that the iterations skip over it and proceed to the load buses and the PV bus.

Meanwhile, the general updating formula for a PQ load bus is derived from the power flow equations and the Gauss-Seidel method's formula itself, given the current-voltage relationship from the Y-bus and the complex power injection equations:

$$V_i^{k+1} = \left[\frac{1}{Y_{ii}} \right] \left[\frac{(P_i - Q_i j)}{V_i^{*k}} - \sum_{j \neq i} Y_{ij} V_j \right]$$

Where the voltage V_i^{*k} represents the complex conjugate of the voltage computed for the bus from the previous iteration. By default, this is set to $1.0\angle 0^\circ$ pu to be consistent with standard "flat start" power flow practices; however, the program has been modified to allow user-defined initial voltage guesses to observe differing convergence behaviors. It is also important to note that the k variable represents the iteration number of the Gauss-Seidel method.

The above equation is also used for the PV/generator bus (Bus 2), which is treated as an ideal voltage-controlled source. However, as the voltage magnitude is fixed at the setpoint of 1.02 pu throughout the iterations, only the phasor is extracted from the equation to retain and improve the solution in ideal situations. The reactive power Q of this bus is also different from PQ buses, and is computed using the formula to be used in the voltage equation earlier:

$$Q_2^k = -Im\{V_2^{*k} \times (Y_2 \cdot V^k)\}$$

Furthermore, an acceleration factor is applied in this implementation of the Gauss-Seidel method, written as $\alpha = 1.3$. This is done to each bus voltage update as a form of Successive/Simultaneous Over-Relaxation (SOR) as described in Chapra and Canale's explanation of the aforementioned algorithm (2015). This value is chosen empirically and arbitrarily as per their description, and it is done to accelerate the convergence of this system as it already appears to be convergent without it. This update is illustrated in the voltage equation below:

$$V_i = V_i^k + \alpha(V_i^{k+1} - V_i^k)$$

Lastly, since the Gauss-Seidel method requires a stopping criterion to complete a full implementation without iterating forever, the approximate relative error formula is used, which is the maximum absolute voltage difference divided by the current voltage solution guess across all buses between iterations. This is modified to use the actual error rather than the percent errors computed, resulting in the formula:

$$E_i^k = \left| \frac{V_i^{k+1} - V_i^k}{V_i^{k+1}} \right|$$

In converting and computing the numerical values that are obtained in pu into more realistic systems like watts and SI units, a base power of 100 converts and rounds the calculations for it to transform into a megawatt measurement:

$$P_{actual} = P_{pu} \times S_{base}; Q_{actual} = Q_{pu} \times S_{base}$$

This base power of 100 MVA is commonly used in power system studies as a standard normalization value assuming a near-unity power factor, allowing straightforward interpretation of per-unit quantities as percentages. Furthermore, the

chosen base power results in total system load values that align with realistic residential microgrid scales (~100 MW).

3.4 Results Display

Upon convergence, the program produces two categories of output: a formatted terminal summary printed via the Gauss-Seidel function itself, another specific `print_results()` function, and a four-panel matplotlib figure generated by another `plot_results()` function.

The Gauss-Seidel function specifically prints each iteration and the converged values if and when the stopping criterion is met, tabulated in the terminal of the main implementation. Within each row of the table, The absolute voltage in $|V|$, and phasor angle in degrees, are shown per bus and an additional column is shown for the maximum error obtained in the buses' approximate error calculations per voltage.

Meanwhile, the `print_results()` function additionally does the following: it first iterates over the converged voltage array to print each bus's magnitude and phase angle alongside its bus type for the final output to be emphasized, and it then calls `compute_slack_injection()`, which back-calculates the slack bus complex power using the following equation:

$$S = V_1 \times (Y_1 \cdot V)^*$$

where the asterisk denotes the conjugate of the dot product. The earlier function then prints the resulting real and reactive power injection. Next, it loops over the dictionary returned by `compute_line_flows()` to print the sending-end real and reactive power and the total resistive loss for each of the five lines, computed as the sum of the symmetric powers. The function closes by summing all line losses and printing the total alongside the iteration count.

Lastly, a matplotlib and its sublibrary gridspec is utilized for the `plot_results()` function which arranges four subplots within a single figure using the sublibrary and the overall plotting functionality of matplotlib. The first panel renders the convergence curve by plotting the approximate errors' list on a semi-logarithmic axis, with a dashed horizontal line marking the tolerance threshold. The second panel draws a bar chart of the converged voltage magnitudes at each bus, annotating each bar with its numerical value and overlaying reference lines at 0.95 pu and 1.05 pu. The third panel, meanwhile, constructs a polar phasor diagram by extracting the angle and magnitude of each complex bus voltage and drawing annotated arrows from the origin. Lastly, the

final panel draws a simplified one-line diagram of the microgrid by plotting bus nodes at a graph-like drawing and connecting them with lines, labeling each line with the computed sending-end real power flow from the flows dictionary.

3.5 *User Input Additions and Modifications*

To fully maximize the operations of the Gauss-Seidel-based voltage estimation program, a user-input option is also created for the expansion of the study. It fully realizes the importance of the Gauss-Seidel method as a viable and advantageous voltage calculator while still retaining its small-scale advantage that is perfect for microgrid-based systems.

Specifically, the program has been upgraded to support dynamic network topologies. While the default configuration is the four-bus system described earlier, users can now input the desired number of buses, with a hardcoded operational limit set between four to six buses. This limitation ensures that the mathematical model remains within the intended low-voltage, small-scale microgrid scope and guarantees that the terminal displays and graphical one-line diagrams remain legible and uncluttered.

Alongside defining the total bus count, users are prompted to enter the characteristics of any additional buses (e.g., scheduled real and reactive power demands for new PQ load centers) as well as the complex impedances of the new transmission lines connecting them. Furthermore, the system allows for the manual input of initial voltage guesses. Instead of strictly relying on the traditional flat start of $1.0\angle 0^\circ$ pu, users can input custom initial magnitudes and angles for the unknown buses. This interactive feature allows engineers to deliberately skew the starting parameters to test how different initial conditions affect the convergence speed, stability, and iteration count of the Gauss-Seidel algorithm, making the tool highly adaptable to various hypothetical microgrid scenarios.

The user-input extension of the program is added across six sequential cells (Cells 9 to 14), each handling a distinct phase of configuration. Rather than replacing the core solver and display functions defined in Cells 1 to 6, these cells exclusively handle data collection and then pass the resulting structures directly into the same `build_ybus()`, `gauss_seidel_power_flow()`, `compute_line_flows()`, `print_results()`, and `plot_results()` functions. This design ensures that all validation, convergence logic, and output formatting remain consistent between the hardcoded four-bus example and any user-defined system.

Cell 9 establishes two shared helper utilities and performs the initial system sizing step. The `_parse_complex()` helper wraps Python's native `complex()` constructor

with a descriptive error message, allowing the program to accept any standard complex literal such as $0.02+0.06j$ or $-0.55-0.18j$ entered at the prompts that follow. The `_prompt_voltage()` helper function additionally supports a default-value fallback, returning a pre-specified complex voltage unchanged if the user presses Enter without typing anything. Following these definitions, the user is prompted to specify the total number of buses in the system, which is constrained to integers between four and six, inclusive. This range is deliberately chosen to preserve the small-scale advantage of the Gauss-Seidel method while still allowing a meaningful expansion beyond the four-bus base example.

Cell 10 handles bus type assignment. Bus 1 is unconditionally designated as the slack bus, consistent with the mandatory single-reference-bus requirement of any load flow formulation. For each remaining bus, the user is prompted to classify it as either a PV (generator) bus or a PQ (load) bus by entering `pv` or `pq` respectively. The assignments are collected into a dictionary variable called `bus_types_map` that is subsequently threaded through the solver and print functions, replacing the hardcoded type labels used in the four-bus example.

Cell 11, meanwhile, collects the network topology and transmission line parameters. The user enters one line at a time in the format `{from_bus} {to_bus} {R+Xj}`, each variable separated by a space, where the impedance value is parsed using `_parse_complex()`. Input continues until the user submits a blank entry, at which point the dictionary `user_impedances` is finalized. The cell enforces several validity conditions on each entry: both bus numbers must fall within the valid range and must differ from each other, and the impedance value must be nonzero to avoid a singularity when computing the line admittance. Each accepted entry is immediately echoed back to the user with its stored key and value. A minimum of one line must be entered before the input loop may terminate.

Cell 12 collects scheduled complex power injections for all non-slack buses. The prompt format is adapted to each bus type: PV buses are asked only for their scheduled active power P in per-unit, since the reactive power is recomputed at every iteration from the current voltage estimate as per the program's processing flow, while PQ buses are prompted for the full complex injection $P + jQ$. All values are parsed through `_parse_complex()`, and the real and imaginary parts are separated and stored in the `user_bus_power` dictionary under the convention that the slack bus entry remains `(None, None)`, to be back-calculated after convergence.

Cell 13 gathers voltage specifications. The slack bus is prompted for a full complex voltage with a default of $1.05+0j$, consistent with the hardcoded example's

above-nominal reference. PV buses accept a floating-point magnitude with a default of 1.02 as a remark to the original example, and PQ buses accept a complex flat-start initial guess defaulting to $1.0+0j$, where most Gauss-Seidel implementations start with if no specified initial values are assigned. All prompts are routed through `_prompt_voltage()` or `direct_float()` conversion, with automatic fallback to the stated default on a blank entry. The collected values are stored in `user_bus_voltage_spec`, mirroring the structure of `bus_voltage_spec` in the fixed example.

Cell 14 executes the analysis. It builds the Y-bus matrix via `build_ybus` (`user_impedances`, `n_buses=n_buses`), prints the resulting admittance matrix row by row, then calls `gauss_seidel_power_flow()`, `compute_line_flows()`, `print_results()`, and `plot_results()` in the same sequence used by the fixed four-bus main cell, passing the user-supplied dictionaries and the `bus_types_map` in place of their hardcoded equivalents.

To illustrate the expanded capability, the user-input cells were exercised on a six-bus microgrid system. The system was specified with one slack bus (Bus 1), one PV generator bus (Bus 2), and four PQ load buses (Buses 3 to 6), connected by nine transmission lines. The resulting 6×6 Y-bus admittance matrix, printed as part of the program output, is shown in Figure 6. Notably, several off-diagonal entries in the matrix are zero, reflecting the fact that not all buses share a direct transmission line; for instance, Bus 2 has no direct connection to Buses 4 or 6, and Bus 3 connects exclusively to Buses 1, 2, and 4. These structural zeros are automatically produced by the `build_ybus()` function based solely on the user-supplied line list, requiring no additional specification from the user.

3.6 User Input Sample

To fully discuss the capabilities of the expanded user-input subprocess, the following sample inputs are given to the set of cells. The results obtained shall also be discussed in a further section of this paper to prove the proficiency of the software and the Gauss-Seidel method in computing and estimating voltages across a relatively larger microgrid system while still being of a small size where the aforementioned method is most advantageous. It should be noted that the initial voltages guessed are still the same as the initial voltages in the previous example, as this is to put the results into similar conditions.

Line	Resistance R (pu)	Reactance X (pu)	Impedance Z
Bus 1 to Bus 2	0.02	0.06	$0.02 + 0.06j$
Bus 2 to Bus 3	0.04	0.12	$0.04 + 0.12j$
Bus 3 to Bus 4	0.06	0.18	$0.06 + 0.18j$
Bus 4 to Bus 5	0.05	0.15	$0.05 + 0.15j$
Bus 5 to Bus 6	0.04	0.12	$0.04 + 0.12j$
Bus 6 to Bus 1	0.03	0.10	$0.03 + 0.10j$
Bus 1 to Bus 3	0.08	0.24	$0.08 + 0.24j$
Bus 2 to Bus 5	0.06	0.18	$0.06 + 0.18j$
Bus 1 to Bus 4	0.01	0.05	$0.01 + 0.05j$

Table 3. Complex impedances given to sample user input

Bus	Power Injections (in p.u.)
1	N/A (slack bus)
2	0.55 (PV bus; no Q)
3	$-0.45-0.15j$
4	$-0.50-0.20j$
5	$-0.35-0.10j$
6	$-0.40-0.12j$

Table 4. Bus power injections inputted for the 6-bus example

	Bus 1	Bus 2	Bus 3	Bus 4	Bus 5	Bus 6
Bus 1	+12.85 -47.16j	-5.00 +15.00j	-1.25 +3.75j	-3.85 +19.23j	0	-2.75 +9.17j
Bus 2	-5.00 +15.00j	+9.17 -27.50j	-2.50 +7.50j	0	-1.67 +5.00j	0
Bus 3	-1.25 +3.75j	-2.50 +7.50j	+5.42 -16.25j	-1.67 +5.00j	0	0
Bus 4	-3.85 +19.23j	0	-1.67 +5.00j	+7.51 -30.23j	-2.00 +6.00j	0
Bus 5	0	-1.67 +5.00j	0	-2.00 +6.00j	+6.17 -18.50j	-2.50 +7.50j
Bus 6	-2.75 +9.17j	0	0	0	-2.50 +7.50j	+5.25 -16.67j

Table 5. Y-Bus matrix of the input

3.7 Program Optimization

Several optimizations were incorporated into the Python implementation to improve both computational efficiency and numerical performance.

First, the Y-bus matrix is constructed once prior to the iterative loop and stored as a NumPy complex array, which avoids repeated recomputation of admittances during iteration. The use of NumPy's vectorized operations, particularly the row-vector dot product $Y[i] @ V$ for computing bus currents during the PV bus reactive power update, replaces explicit Python loops and yields measurable speedup for matrix-vector multiplications.

Second, the Successive Over-Relaxation (SOR) technique with $\alpha = 1.3$ is embedded directly within the voltage update step, reducing the number of iterations required for convergence from approximately 18 (without acceleration, $\alpha = 1.0$) to 10 iterations. This represents a roughly 55.56% reduction in iteration count, with negligible additional computation per step.

Finally, Python's warnings module is configured to suppress runtime warnings, and all intermediate results such as voltage history and mismatch deltas are stored in pre-allocated lists rather than growing data structures that would require frequent memory reallocation.

3.8 Considerations for Alternative Design

While the Gauss-Seidel method was selected as the primary solver for this project, several alternative approaches could be considered for comparison or future extension of this solution.

The Newton-Raphson method is the most widely used alternative for power flow analysis in professional-grade simulation tools. Unlike Gauss-Seidel, which updates one bus at a time, Newton-Raphson solves the full nonlinear power mismatch equations simultaneously using a Jacobian matrix. This typically results in quadratic convergence, meaning the error decreases quadratically per iteration, reaching the same solution in far fewer iterations. However, Newton-Raphson requires the construction and factorization of a Jacobian matrix at each step, making each iteration computationally heavier. For small systems such as this four-bus microgrid, the overhead is negligible, but for large-scale grids with hundreds of buses, Newton-Raphson's superior convergence rate makes it clearly preferable.

The Fast Decoupled Load Flow (FDLF) method offers another alternative by exploiting the weak coupling between real power and voltage magnitude, and between reactive power and voltage angle, which holds approximately in high-voltage transmission systems. This decoupling allows the Jacobian to be simplified into two smaller, constant matrices, substantially reducing the per-iteration cost. However, this approximation is less valid in low-voltage distribution microgrids with high resistance-to-reactance ratios such as the one modeled here, where real and reactive power flows are more tightly coupled. As such, FDLF would be less accurate and potentially less reliable for this specific system.

In terms of modifications concerning the Gauss-Seidel structure itself, a possible modification is the replacement of the SOR acceleration with an adaptive relaxation scheme, where the acceleration factor α is dynamically adjusted based on the convergence rate observed between iterations. Such adaptive methods can improve robustness for systems where a fixed α might cause oscillations or slow convergence in certain operating conditions.

Finally, for island microgrids with distributed energy resources, droop-based power sharing models could be integrated into the bus specifications, replacing the fixed PV bus assumptions with dynamic generator models. This would allow the simulation to capture transient power-sharing behavior among distributed generators rather than assuming ideal voltage regulation, making the model more representative of real microgrid operation.

IV. Results, Analysis, and Discussion

The power flow analysis of the four-to-six bus microgrid was executed using the Gauss-Seidel iterative method. This section evaluates the computational performance, nodal voltage stability, and the physical integrity of the resulting power distribution in Figure 3. The results of this study validate the application of the Gauss-Seidel iterative method as a robust and efficient tool for voltage estimation within small-scale power systems. While modern power systems research frequently prioritizes high-order algorithms for expansive transmission networks, this analysis demonstrates that the Gauss-Seidel method remains a more practical choice for the decentralized, small-scale systems of four to six buses architectures that are more pushed in localized energy systems and residential communities in the Philippines (Microgrid Systems Act, 2022).

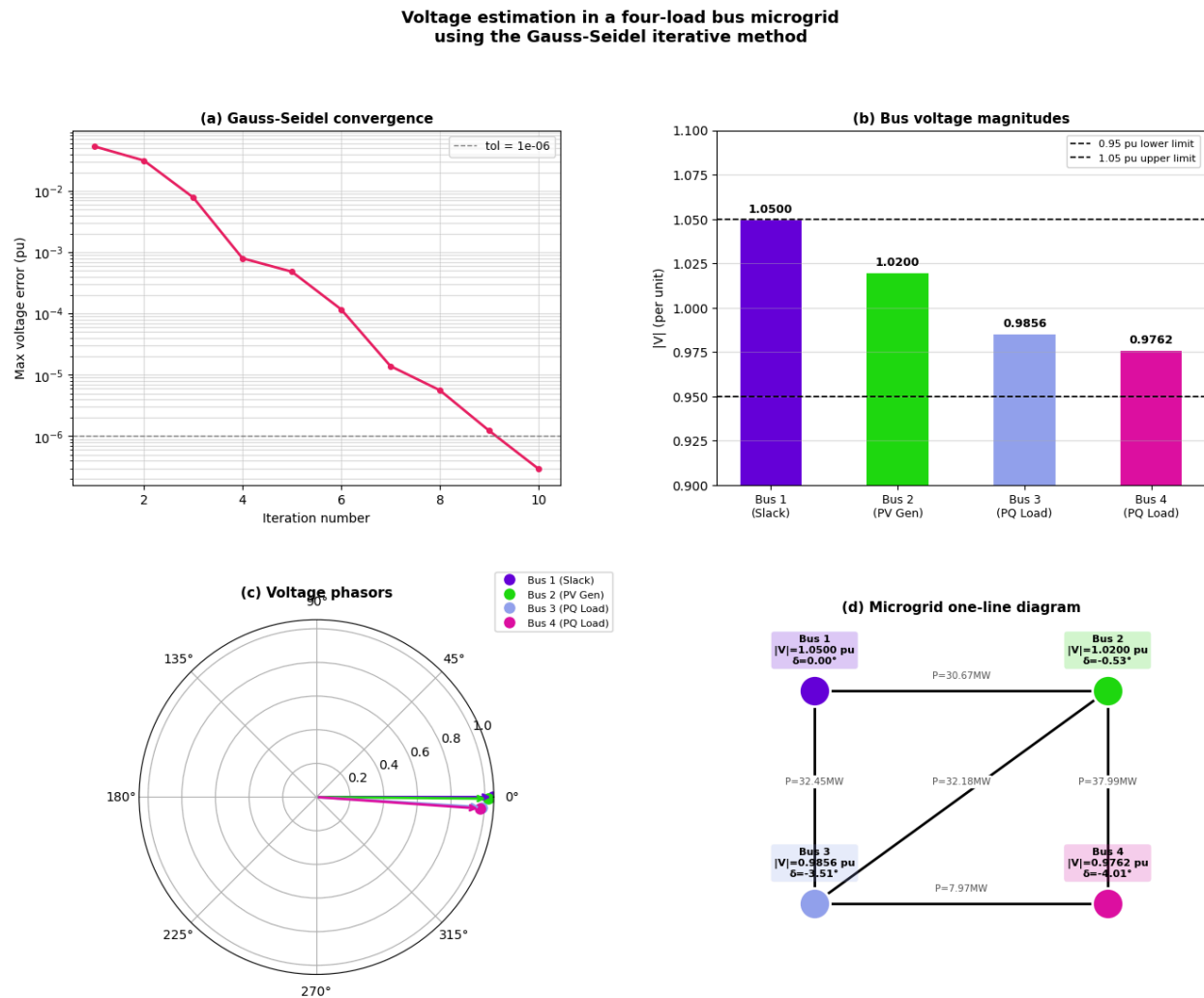


Figure 3. Voltage estimation in a four-load bus microgrid using the Gauss-Seidel iterative method. (a) Max approximate relative voltage error per iteration demonstrating convergence within 10 iterations. (b) Computed bus voltage magnitudes compared to

acceptable limits of $[0.95, 1.05]$ pu. (c) Voltage phasors in polar coordinates visualizing relative magnitudes and angles. (d) Microgrid one-line diagram overlaid with solved voltage profiles ($|V|$ in per unit, δ in degrees) and real power line flows (P in MW).

Gauss-Seidel Power Flow on a Four-Bus Microgrid
NUMMETS-E32/EQ2 | Group 7

Y-Bus Matrix:
['+6.2500-18.7500j', '-5.0000+15.0000j', '-1.2500+3.7500j', '+0.0000+0.0000j']
['-5.0000+15.0000j', '+8.3333-25.0000j', '-1.6667+5.0000j', '-1.6667+5.0000j']
['-1.2500+3.7500j', '-1.6667+5.0000j', '+5.0788-15.7770j', '-2.1622+7.0270j']
['+0.0000+0.0000j', '-1.6667+5.0000j', '-2.1622+7.0270j', '+3.8288-12.0270j']

Iter	Bus 1		Bus 2		Bus 3		Bus 4		max dV/V
	V (pu)	angle (°)	V (pu)	angle (°)	V (pu)	angle (°)	V (pu)	angle (°)	
0 (flat)	1.050000	0.0000	1.020000	0.0000	1.000000	0.0000	1.000000	0.0000	--
1	1.050000	0.0000	1.020055	1.2429	0.997315	-1.5713	0.980469	-2.8065	5.33e-02
2	1.050000	0.0000	1.020047	-0.0844	0.986263	-3.2427	0.976119	-3.9461	3.14e-02
3	1.050000	0.0000	1.019993	-0.5348	0.985583	-3.5574	0.976222	-4.0751	7.86e-03
4	1.050000	0.0000	1.020002	-0.5558	0.985605	-3.5384	0.976139	-4.0296	7.98e-04
5	1.050000	0.0000	1.019999	-0.5377	0.985564	-3.5108	0.976149	-4.0135	4.83e-04
6	1.050000	0.0000	1.020000	-0.5310	0.985599	-3.5079	0.976180	-4.0127	1.17e-04
7	1.050000	0.0000	1.020000	-0.5311	0.985610	-3.5084	0.976179	-4.0134	1.39e-05
8	1.050000	0.0000	1.020000	-0.5313	0.985607	-3.5087	0.976177	-4.0135	5.64e-06
9	1.050000	0.0000	1.020000	-0.5314	0.985606	-3.5087	0.976177	-4.0135	1.23e-06
10 [done]	1.050000	0.0000	1.020000	-0.5314	0.985606	-3.5087	0.976177	-4.0135	2.94e-07

Converged at iteration 10 (e% = 2.94e-07)

Voltage estimation results

Bus voltages (converged solution):

bus	V (pu)	angle (°)	type
1	1.050000	0.000000	slack
2	1.020000	-0.531382	pv (gen)
3	0.985606	-3.508720	pq (load)
4	0.976177	-4.013511	pq (load)

Slack bus injection:
P = 63.1171 MW
Q = 60.5184 MVar

Line power flows (base = 100 MVA):

line	P_ij(MW)	Q_ij(MVar)	loss_MW
1-2	30.6721	42.3527	0.4961
1-3	32.445	18.1657	1.0033
2-3	32.1817	9.5166	0.6495
2-4	37.9943	13.1895	0.9328
3-4	7.9739	4.7239	0.0354

Total real power losses: 3.1171 MW
Iterations to convergence: 10

Figure 4. Gauss-Seidel Power Flow on a Four-Bus Microgrid Program Run Results

4.1.1 Gauss-Seidel Iteration Analysis for 4-bus example

The computational efficiency of the power flow solution is heavily dictated by the convergence characteristics of the iterative process. As shown in Figure 3(a), the maximum voltage mismatch is defined as the absolute difference between successive iterations $|V_i^{k+1} - V_i^k|$ and exhibits a distinct near-exponential decay.

The implementation utilized a Successive Over-Relaxation (SOR) factor of $\alpha = 1.3$ to accelerate the convergence rate. Mathematically, the SOR modification replaces the standard Gauss-Seidel update with a weighted average:

$$V_i = V_i^k + \alpha(V_i^{k+1} - V_i^k)$$

By selecting $\alpha = 1.3$, the algorithm effectively pushes the solution further in the direction of the correction, significantly reducing the spectral radius of the iteration matrix. In practical terms, this reduced the required iterations from 18 or more (in non-accelerated cases or higher accelerated values) down to a streamlined 10 iterations. Higher and lower iterations indicated in testing, including underrelaxation acceleration values, showed more convergence iterations, as shown below:

Converged at iteration 26 (e% = 9.63e-07)					Converged at iteration 18 (e% = 6.06e-07)					Converged at iteration 21 (e% = 6.27e-07)					Converged at iteration 45 (e% = 8.31e-07)				
Voltage estimation results					Voltage estimation results					Voltage estimation results					Voltage estimation results				
Bus voltages (converged solution):					Bus voltages (converged solution):					Bus voltages (converged solution):					Bus voltages (converged solution):				
bus	V (pu)	angle (°)	type		bus	V (pu)	angle (°)	type		bus	V (pu)	angle (°)	type		bus	V (pu)	angle (°)	type	
1	1.050000	0.000000	slack		1	1.050000	0.000000	slack		1	1.050000	0.000000	slack		1	1.050000	0.000000	slack	
2	1.020000	-0.531300	PV (gen.)		2	1.020000	-0.531351	pv (gen)		2	1.020000	-0.531373	pv (gen)		2	1.020000	-0.531406	pv (gen)	
3	0.985606	-3.508645	PQ (load)		3	0.985606	-3.508677	pq (load)		3	0.985606	-3.508697	pq (load)		3	0.985606	-3.508706	pq (load)	
4	0.976177	-4.013397	PQ (load)		4	0.976177	-4.013473	pq (load)		4	0.976177	-4.013505	pq (load)		4	0.976177	-4.013522	pq (load)	

Figure 5. Screenshots of excerpts using different acceleration values on the Gauss-Seidel implementation results. (a) Alpha = 0.8 (underrelaxation) (b) Alpha = 1.0 (no SOR acceleration); (c) Alpha = 1.6; (d) Alpha = 1.8

The "regression-like" linearity of the error plot on a semi-logarithmic scale indicates a stable and consistent convergence path. This behavior is particularly vital for residential microgrids where the high resistance-to-reactance (R/X) ratios of low-voltage lines can often lead to numerical instability in more rigid algorithms. The final residual error of 2.94×10^{-7} provides a high-confidence estimate, ensuring that the nodal voltages discussed in Section 4.2 are not merely numerical artifacts but represent a true steady-state equilibrium.

4.1.2 Nodal Voltage Analysis

The steady-state voltage profiles for the microgrid are summarized in the table below. The results indicate that all bus voltages remain within the conventional operational envelope of [0.95, 1.05] pu, validated visually in the bar chart comparison of Figure 3(b).

Bus Type	Identifier	Voltage Magnitude (pu)	Phase Angle (degrees)
Slack Bus	Bus 1	1.05	0.00
PV Bus	Bus 2	1.02	-0.531
PQ Bus	Bus 3	0.985606	-3.508
PQ Bus	Bus 4	0.976177	-4.015

Table 6. 4-bus example Gauss-Seidel results

The solved network state, including detailed nodal voltage values annotated directly on the microgrid structure, is presented in Figure 3(d). The observed voltage gradient is characterized by a progressive decline in magnitude from the generation sources (Buses 1 and 2) toward the load centers (Buses 3 and 4). The observed voltage gradient also aligns with the expected voltage drops across line impedances. Furthermore, both the tabular data and the visual representation in Figures 3(c) and 3(d) phase angles exhibit a monotonic decrease relative to the slack bus. In accordance with power system theory, this negative angular shift confirms the unidirectional flow of real power from the reference bus and distributed generators toward the downstream loads.

4.1.3 Power Balance and System Losses

To validate the numerical solution, a power balance was performed. The slack bus injection was calculated at 63.1171 MW and 60.5184 MVar. When combined with the 40 MW generation at the PV bus, the total system real power generation P_{gen} equals 103.1171 MW. This value precisely satisfies the power balance equation:

$$P_{gen} = P_{load} + P_{loss}$$

Where $P_{load} = 100$ MW and $P_{loss} = 3.1171$ MW. The transmission losses account for approximately 3.02% of the total generation, a figure representative of distribution-level microgrids where the R/X ratio is typically higher than in high-voltage transmission systems.

A visual overview of the power distribution across the network is provided in Figure 3(d), overlaying real power (P in MW) values and flow directions on the system lines. The distribution of power across the network reveals a well-balanced load-sharing configuration. The primary power corridor was identified between Bus 2 and Bus 4 (37.99 MW), highlighting the critical role of the PV bus in supporting local demand. Bus 3 receives nearly equal contributions from both the slack bus (32.45 MW) and the PV bus (32.18 MW), indicating a redundant and reliable supply topology. The relatively low power transfer between Bus 3 and Bus 4 (7.97 MW) suggests a weaker electrical coupling, implying that these buses operate largely as independent load centers within the microgrid architecture.

4.2.1 User Input version results

Voltage estimation in a 6-bus microgrid using the Gauss-Seidel iterative method

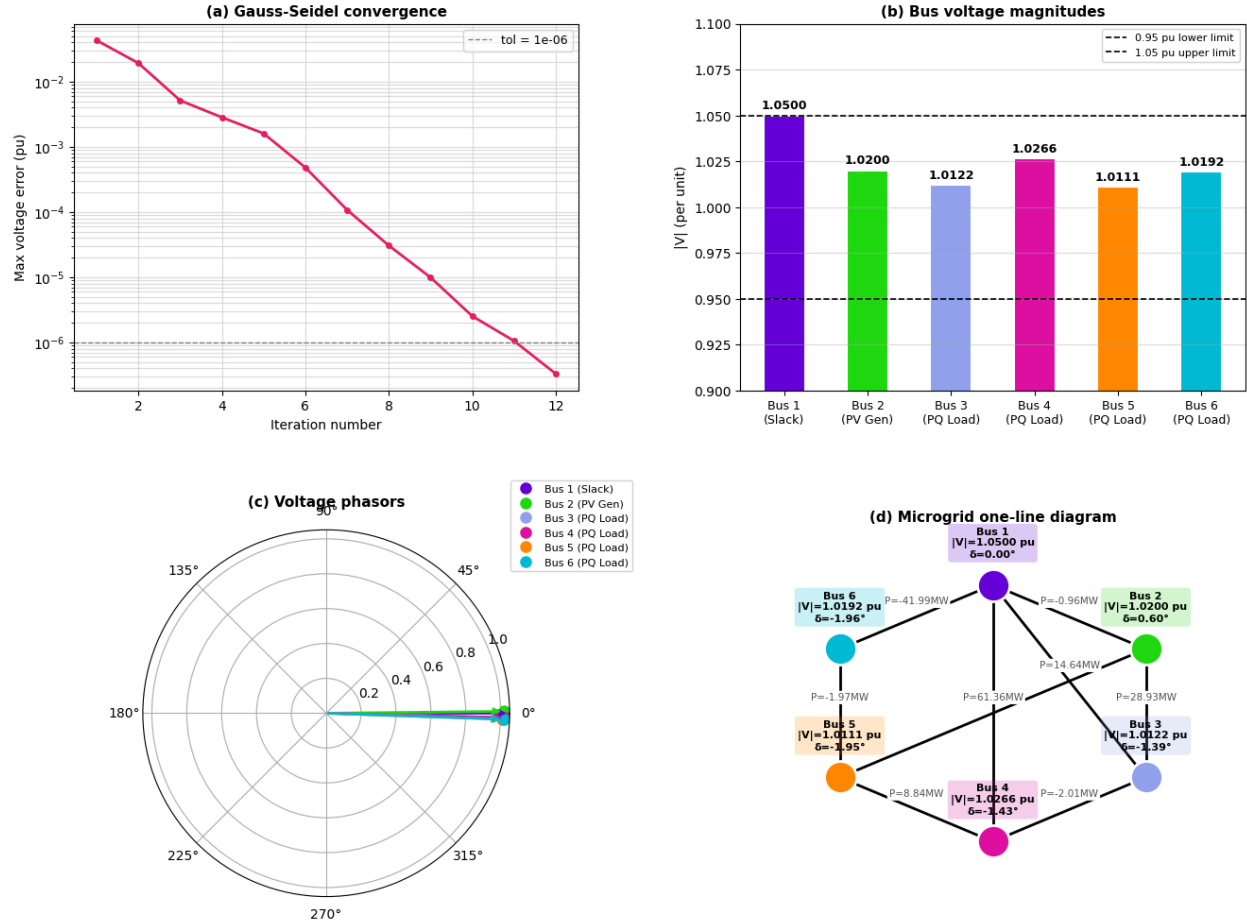


Figure 5. Voltage estimation in a six-load bus microgrid using the Gauss-Seidel iterative method's user input set of cells. (a) Max approximate relative voltage error per iteration demonstrating convergence within 12 iterations. (b) Computed bus voltage magnitudes compared to acceptable limits of $[0.95, 1.05]$ pu. (c) Voltage phasors in polar coordinates visualizing relative magnitudes and angles. (d) Microgrid one-line diagram overlayed with solved voltage profiles ($|V|$ in per unit, δ in degrees) and real power line flows (P in MW).

```
Running Gauss-Seidel Power Flow with user-supplied data
-----
Y-Bus Matrix:
[ '+12.8484-47.1551j', '-5.0000+15.0000j', '-1.2500+3.7500j', '-3.8462+19.2308j', '+0.0000+0.0000j', '-2.7523+9.1743j' ]
[ '-5.0000+15.0000j', '+9.1667-27.5000j', '-2.5000+7.5000j', '+0.0000+0.0000j', '-1.6667+5.0000j', '+0.0000+0.0000j' ]
[ '-1.2500+3.7500j', '-2.5000+7.5000j', '+5.4167-16.2500j', '-1.6667+5.0000j', '+0.0000+0.0000j', '+0.0000+0.0000j' ]
[ '-3.8462+19.2308j', '+0.0000+0.0000j', '-1.6667+5.0000j', '+7.5128-30.2308j', '-2.0000+6.0000j', '+0.0000+0.0000j' ]
[ '+0.0000+0.0000j', '-1.6667+5.0000j', '+0.0000+0.0000j', '-2.0000+6.0000j', '+6.1667-18.5000j', '-2.5000+7.5000j' ]
[ '-2.7523+9.1743j', '+0.0000+0.0000j', '+0.0000+0.0000j', '+0.0000+0.0000j', '-2.5000+7.5000j', '+5.2523-16.6743j' ]

-----
Iter      Bus 1      Bus 2      Bus 3      Bus 4      Bus 5      Bus 6      max|dV/V|
      |V|(pu) angle (°) |V|(pu) angle (°) |V|(pu) angle (°) |V|(pu) angle (°) |V|(pu) angle (°) |V|(pu) angle (°)
-----
0 (flat)  1.050000  0.0000  1.020000  0.0000  1.000000  0.0000  1.000000  0.0000  1.000000  0.0000  1.000000  0.0000  --
1         1.050000  0.0000  1.020074  1.4384  1.005341  -0.7658  1.029385  -1.2793  1.005732  -1.1805  1.022326  -2.1472  4.30e-02
2         1.050000  0.0000  1.019995  0.7377  1.015480  -1.4469  1.024894  -1.2928  1.013822  -2.1896  1.019838  -2.0291  1.93e-02
3         1.050000  0.0000  1.020003  0.5253  1.010582  -1.3473  1.027551  -1.5285  1.011010  -1.9785  1.018908  -1.9482  5.15e-03
4         1.050000  0.0000  1.019999  0.6079  1.013027  -1.4328  1.026468  -1.4165  1.010909  -1.9279  1.019138  -1.9457  2.84e-03
5         1.050000  0.0000  1.020000  0.5883  1.011878  -1.3678  1.026499  -1.4266  1.011096  -1.9538  1.019189  -1.9612  1.61e-03
6         1.050000  0.0000  1.020000  0.6018  1.012238  -1.3865  1.026629  -1.4336  1.011112  -1.9519  1.019176  -1.9551  4.82e-04
7         1.050000  0.0000  1.020000  0.5957  1.012183  -1.3865  1.026581  -1.4309  1.011083  -1.9503  1.019165  -1.9561  1.08e-04
8         1.050000  0.0000  1.020000  0.5968  1.012178  -1.3848  1.026586  -1.4310  1.011088  -1.9510  1.019171  -1.9562  3.06e-05
9         1.050000  0.0000  1.020000  0.5970  1.012183  -1.3853  1.026587  -1.4312  1.011089  -1.9509  1.019170  -1.9561  9.91e-06
10        1.050000  0.0000  1.020000  0.5968  1.012182  -1.3853  1.026587  -1.4311  1.011089  -1.9509  1.019170  -1.9561  2.53e-06
11        1.050000  0.0000  1.020000  0.5968  1.012182  -1.3852  1.026587  -1.4312  1.011089  -1.9509  1.019170  -1.9561  1.05e-06
12 [done]  1.050000  0.0000  1.020000  0.5969  1.012182  -1.3852  1.026587  -1.4312  1.011089  -1.9509  1.019170  -1.9561  3.29e-07

-----
Converged at iteration 12 (e% = 3.29e-07)
-----
```

```
Voltage estimation results

Bus voltages (converged solution):

bus      |V| (pu)      angle (°)      type
-----
1         1.050000      0.000000      slack
2         1.020000      0.596859      PV (Gen.)
3         1.012182     -1.385247      PQ (Load)
4         1.026587     -1.431161      PQ (Load)
5         1.011089     -1.950878      PQ (Load)
6         1.019170     -1.956126      PQ (Load)

Slack bus injection:
P = 117.6321 MW
Q = 122.4974 MVar

Line power flows (base = 100 MVA):

line      P_ij(MW)      Q_ij(MVar)      loss_MW
-----
1+2       -0.9558       52.9155       0.5081
2+3       28.9298      -2.4829       0.3241
3+4       -2.0137      -7.4288       0.0347
4+5       8.8396       7.6887       0.0651
5+6       -1.972       -6.1519       0.0163
6+1       -41.9883     -18.2009       0.6049
1+3       14.6372      11.7958       0.2564
2+5       24.6066      -2.5861       0.353
1+4       61.3576      37.569        0.4695

Total real power losses: 2.6321 MW
Iterations to convergence: 12
```

Figure 6. Six-bus microgrid results using user input

4.2.2 Gauss-Seidel Iteration Analysis

As shown in Figure 5(a) and in the Figure 6 screenshots, the six-bus system converges within 12 iterations under the same SOR acceleration factor of $\alpha = 1.3$ used in the four-bus example, which were kept constant in this case as this output is a sample version for users. The two additional iterations relative to the four-bus case are consistent with expectations, as the expanded network introduces additional off-diagonal coupling terms in the Y-bus matrix, slightly increasing the spectral radius of the iteration operator. Nevertheless, the error trajectory on the semi-logarithmic scale remains approximately linear, confirming the same stable, near-exponential decay behavior observed in the four-bus case. The final residual error of 3.29×10^{-7} falls below the prescribed tolerance of 1×10^{-6} , establishing that the converged voltage profile represents a real equilibrium rather than an erroneous chance or an early stoppage.

4.2.3 Nodal Voltage Analysis

Bus Type	Identifier	Voltage Magnitude (pu)	Phase angle (degrees)
Slack	1	1.05	0
PV	2	1.02	0.596859
PQ	3	1.012182	-1.385247
PQ	4	1.026587	-1.431161
PQ	5	1.011089	-1.950878
PQ	6	1.019170	-1.956126

Table 7. Voltage and phasor results for 6-bus user input

The converged bus voltages for the six-bus system are summarized in the table within Figure 6. All six buses remain within the conventional operational envelope of 0.95-1.05 pu that was predefined from the earlier result, as validated visually in Figure 5(b). A notable distinction from the four-bus result is that all buses in the six-bus system converge above 1.0 pu, with the load buses ranging from 1.011 pu (Bus 5) to 1.027 pu (Bus 4).

This comparatively elevated voltage profile reflects the characteristics of the user-specified network, where the supply infrastructure is sufficiently strong and the load buses are positioned in close electrical proximity to generation sources such that

minimal voltage depression occurs. Bus 2, the PV generator bus, exhibits a small positive phase angle of $+0.6^\circ$ relative to the slack bus, in contrast to the negative phase angles observed at all non-slack buses in the four-bus case.

This positive angular displacement indicates that the PV bus leads the slack bus in phase, which is consistent with a generator operating under active injection conditions with a relatively low-impedance path connecting it to the slack reference. Buses 3 through 6 exhibit progressively negative phase angles, reaching -1.956° at Bus 6, which correctly reflects the direction of active power flow away from the generation sources toward the load centers.

4.2.4 Power Balance and System Losses

The back-calculated slack bus injection yields $P = 117.6321$ MW and $Q = 122.4974$ MVar. When combined with the active power contribution of the PV generator bus, the total generation satisfies the power balance equation $P_{\text{gen}} = P_{\text{load}} + P_{\text{loss}}$, where total transmission losses are computed as 2.6321 MW.

The six-bus system's losses account for a smaller fraction of its total generation compared to the four-bus system's 3.02%, attributable to the expanded supply topology distributing current across a greater number of parallel paths and thereby reducing the I^2R dissipation on any individual line. This result reinforces the well-established principle that network redundancy in meshed microgrids improves both efficiency and reliability.

The graph-like diagram in Figure 5(d) reveals a more distributed power flow pattern relative to the four-bus case, owing to the larger number of interconnections in the user-defined topology. The primary supply corridor in the six-bus network is the direct path from the slack bus to Bus 4, carrying 61.36 MW and serving as the dominant link between the reference source and the mid-network load cluster. Bus 2 fulfills a supplementary but significant supply role, dispatching power toward Buses 3 and 5 with 28.93 MW and 24.61 MW respectively, distributing the generator's contribution across two distinct load branches. The Bus 5 to Bus 6 link carries a comparatively modest flow, while Bus 6 receives power primarily via its direct connection to Bus 1, suggesting that Bus 6 is well-served by the reference source without needing to rely on longer paths through the network interior. The relatively even distribution of non-negligible flows across multiple corridors, rather than concentration in a single dominant path, indicates a well-balanced user-defined topology with no single point of critical loading, which is a desirable characteristic for a resilient residential microgrid.

V. Conclusions

The findings presented and drawn from this paper demonstrate that the Gauss-Seidel iterative method is a highly effective computational engine for small-scale microgrid analysis. While other methods, like the Newton-Raphson method, may computationally arrive faster with quadratic convergence rates and other efficient mathematical choices, their requirements are often complex and introduce unnecessary computational overhead, like the aforementioned Jacobian matrix requirement for a small four- to six-bus system. Alternatively, the Gauss-Seidel method provides lower overhead that is perfectly suited for the expanding sector of decentralized energy infrastructures.

Furthermore, the implementation of Gauss-Seidel raises an important distinction in implementing engineering solutions based on risk, cost, and other parameters indirectly affecting the problem's scale and its needs. Gauss-Seidel, despite its mathematical primitiveness, is still invariably useful in problems at this scale and can perform better in comparison to other iterative or matrix-based systems of linear equation solving algorithms that require complex components to tie in. By upgrading the program to accept user inputs, allowing for the dynamic addition of extra buses (within a controlled four- to six-bus limit) and the testing of custom initial voltage guesses, the tool evolves from a static computation into a highly adaptable testing environment.

This interactive approach enables engineers and community operators to simulate varying network topologies and starting conditions to make informed decisions regarding reactive power compensation, phase balancing, and protection coordination. In the context of the Philippines' growing reliance on localized grids to overcome logistical hurdles, the ability to perform accurate, user-customizable estimations with minimal hardware requirements is a significant operational advantage.

To build upon the foundational model established and address the inherent limitations of a basic Gauss-Seidel implementation, future iterations could investigate the use of dynamic acceleration factors. The implementation of a requirement that adjusts the alpha/relaxation multiplier based on the rate of change in the mismatch residual could further optimize convergence speed, especially when analyzing new, user-defined load configurations (Wang et al., 2025).

Moreover, the algorithm should be expanded to include PV to PQ bus switching to allow the model to simulate scenarios where a generator reaches its reactive power ceiling. This complexity builds upon the effectivity of Gauss-Seidel as a voltage calculation tool in microgrids, as decisions to sacrifice voltage regulation to maintain stability, which is a common occurrence in solar-heavy microgrids during peak irradiation, are put into the equation more (Hosseinzadeh et al., 2021).

This study reinforces the continued relevance of classical iterative techniques such as the Gauss-Seidel method. Through its flexible implementation on smaller scales, utilizing a four-bus baseline that can adaptively scale up to six loads, numerical method applications prove that accessible, simple frameworks can still generate sophisticated, highly accurate, and customizable solutions.

References

- Center for Climate and Energy Solutions. (n.d.). Microgrids. <https://www.c2es.org/content/microgrids/>
- Chiu, R. A. (2026). 7 Bohol island barangays to get solar microgrids by 2029. *Philippine Information Agency*. <https://pia.gov.ph/news/7-bohol-island-barangays-to-get-solar-microgrids-by-2029/>
- Critical Power Systems. (2025, May 30). Newton-Raphson vs. Gauss-Seidel: Power System Analysis Explained. Critical Power Systems. <https://cpsystems.net/blog/power-system-analysis-newton-vs-gauss/>
- Department of Energy. (2026, February 18). *Department of Energy accelerates total electrification through hybrid microgrid system project in Quezon province*. <https://doe.gov.ph/articles/3311479--department-of-energy-accelerates-total-electrification-through-hybrid-microgrid-system-project-in-quezon-province>
- Fiasorgbor, N. K. (2023). Investigating the effects of slack bus selection in load flow studies: A comparative analysis of robust and weak grids. *Journal of Power and Energy Engineering*, 11(9). doi: 10.4236/jpee.2023.119001
- Hosseinzadeh, N., Aziz, A., Mahmud, A., Gargoom, A., & Rabbani, M. (2021). Voltage Stability of Power Systems with Renewable-Energy Inverter-Based Generators: A Review. *Electronics*, 10(2), 115. <https://doi.org/10.3390/electronics10020115>
- Mbasso, W. F., Harrison, A., Jangir, P., Dagal, I., Khishe, M., Smerat, A., Chebaane, S., & Saidani, T. (2025). Reliability-conscious power flow optimization in hybrid renewable microgrids: a case study in Sub-Saharan Africa using Gauss-Seidel and metaheuristic techniques. *International Journal of Electrical Power & Energy Systems*, 173, 111350. <https://doi.org/10.1016/j.ijepes.2025.111350>
- National Laboratory of the Rockies (2025). Microgrids. <https://www.nlr.gov/grid/microgrids>

NPTEL. (n.d.) *Power system analysis: Classification of buses*.
https://archive.nptel.ac.in/content/storage2/courses/108104051/chapter_4/4_3a.html

Rep. Act No. 11646 (2022). *Microgrid Systems Act*.

Sabo, A., Kanya, K. I., Shu'aibu, N., Onyema, C., Aliyu, A., Tanko, H., & Kwasau, S. (2023). A review of State-of-the-Art Techniques for Power Flow Analysis. *Journal of Scientific Technology and Engineering Research*.
<https://doi.org/10.53525/jster.1233034>

Sanitha Michail, C., Rashmi, and Ramachandaramurthy, V. K. (2024). Power Flow Analysis in Microgrid Using Gauss-Seidel Method. *2024 First International Conference on Innovations in Communications, Electrical and Computer Engineering (ICICEC)*. doi: 10.1109/ICICEC62498.2024.10808319

Wang, T., Liu, H., & Su, M. (2025). Energy Optimization for Microgrids Based on Uncertainty-Aware Deep Deterministic Policy Gradient. *Processes*, 13(4), 1047. <https://doi.org/10.3390/pr13041047>

Yang, Z., Xie, K., Yu, J., Zhong, H., Zhang, N, and Xia, Q. (2019). A General Formulation of Linear Power Flow Models: Basic Theory and Error Analysis. *IEEE Transactions on Power Systems*, 34(2), pp. 1315-1324. doi: 10.1109/TPWRS.2018.2871182

Appendix A - Python Code

Preliminary setup cell for 4x4

Python

```
# Gauss-Seidel Implementation on a Four-Bus Microgrid
# NUMMETS-EQ2/E32 Project, Group 7

"""
Setup cell for 4x4 example: given data and imported libraries
"""

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from matplotlib.patches import FancyArrowPatch
import warnings
warnings.filterwarnings("ignore")

# bus types:
# bus 1: slack bus (reference, voltage = 1.0, no angle)
# bus 2: generator (PV) bus
# bus 3: load (PQ) bus
# bus 4: load (PQ) bus

# line impedances (resistance + j*reactance) in per-unit
# format: {(from_bus, to_bus): z}
line_impedances = {
    (1, 2): complex(0.02, 0.06),
    (1, 3): complex(0.08, 0.24),
    (2, 3): complex(0.06, 0.18),
    (2, 4): complex(0.06, 0.18),
    (3, 4): complex(0.04, 0.13),
}

# scheduled power injections at each bus (in per-unit)
# positive = generation, negative = load consumption
# format: {bus: (P_scheduled, Q_scheduled)}
bus_power = {
    1: (None, None),      # slack bus: p and q determined by solution
    2: (0.40, 0.30),      # pv bus: small generator with reactive support
    3: (-0.55, -0.18),    # pq bus: residential load
    4: (-0.45, -0.15),    # pq bus: residential load
}
```

```

# voltage specifications per bus
bus_voltage_spec = {
    1: complex(1.05, 0.0), # slack: 1.05 pu, no angle
    2: 1.02,               # pv: |V| = 1.02 pu (angle unknown)
    3: complex(1.0, 0.0),  # initial flat-start guess
    4: complex(1.0, 0.0),  # initial flat-start guess
}

# convergence settings
MAX_ITERATIONS = 100
TOLERANCE = 1e-6      # convergence threshold in per-unit
ACCEL_FACTOR = 1.3     # gauss-seidel acceleration factor (1.0 = no
                        # acceleration)

```

Admittance matrix cell

Python

Admittance matrix building

```

def build_ybus(impedances, n_buses=4):
    """
    Constructs the nodal admittance matrix Y from line impedance data.
    y_bus[i][j] = -y_ij for off-diagonal (mutual admittance)
    y_bus[i][i] = sum of all y_ij connected to bus i (self admittance)
    """
    Y = np.zeros((n_buses, n_buses), dtype=complex)

    for (i, j), z in impedances.items():
        y = 1.0 / z # line admittance = inverse of impedance
        ii, jj = i - 1, j - 1

        # off-diagonal elements (negative mutual admittance)
        Y[ii][jj] -= y
        Y[jj][ii] -= y

        # diagonal elements (self admittance accumulates)
        Y[ii][ii] += y
        Y[jj][jj] += y
    return Y

```

Gauss-Seidel function cell

Python

Gauss-Seidel main function

```
def gauss_seidel_power_flow(Y, bus_power, bus_voltage_spec,
                             bus_types=None,
                             max_iter=MAX_ITERATIONS,
                             tol=TOLERANCE,
                             accel=ACCEL_FACTOR):
    """
    Solves bus voltages using the Gauss-Seidel iterative method and
    prints a table showing the voltages per bus and the convergence error
    at every iteration.

    bus_types: optional dict {bus_number: 'slack'|'pv'|'pq'}.
                Defaults to the original 4-bus example layout when omitted
                (bus 1 = slack, bus 2 = pv, buses 3-4 = pq).

    Returns the voltage array (V), a list of voltage arrays (history),
    and a list of relative errors (rel_errs).
    """

    n = Y.shape[0]

    # default bus-type map: reproduce the original 4-bus behaviour
    if bus_types is None:
        bus_types = {1: 'slack', 2: 'pv'}
        for b in range(3, n + 1):
            bus_types[b] = 'pq'

    # build initial voltage vector from bus_voltage_spec
    V = np.array([
        complex(bus_voltage_spec[b]) if bus_types[b] != 'pv'
        else complex(bus_voltage_spec[b], 0.0)
        for b in range(1, n + 1)
    ], dtype=complex)

    history = [V.copy()]
    rel_errs = []

    # display table's arbitrary values
    bus_w = 22
    err_w = 12
    iter_w = 12
    sep = '-' * (iter_w + bus_w * n + err_w)
```

```

print(f"\n\t{sep}")
print(f"\t\t{'Iter':^{{iter_w}}}", end='')
for b in range(1, n + 1):
    print(f"{'Bus ' + str(b):^{{bus_w}}}", end='')
print(f"{'max|dV/V|':^{{err_w}}}")

print(f"\t\t{'':^{{iter_w}}}", end='')
for _ in range(n):
    print(f"{'|V|(pu)':^11}{'angle (°)':^11}", end='')
print(f"{'':^{{err_w}}}")
print(f"\t\t{sep}")

print(f"\t\t{'0 (flat)':^{{iter_w}}}", end='')
for v in V:
    print(f"{{abs(v):^11.6f}}{{np.degrees(np.angle(v)):^11.4f}", end='')
print(f"{'--':^{{err_w}}}")
print(f"\t\t{sep}")

for iteration in range(max_iter):
    V_prev = V.copy()

    for i in range(n):
        bus = i + 1
        btype = bus_types[bus]

        if btype == 'slack':          # slack bus voltage is fixed
            continue

        P_sch, Q_sch = bus_power[bus]
        sum_yv = sum(Y[i][j] * V[j] for j in range(n) if j != i)

        if btype == 'pv':            # PV bus, meant for phasor angle calculations
            Q_calc = -np.imag(np.conj(V[i]) * (Y[i] @ V))
            Q_sch = Q_calc
            V_new = (1.0 / Y[i][i]) * ((P_sch - 1j * Q_sch) / np.conj(V[i])
- sum_yv)

            V_new = bus_voltage_spec[bus] * (V_new / abs(V_new))
        else:                        # PQ bus
            V_new = (1.0 / Y[i][i]) * ((P_sch - 1j * Q_sch) / np.conj(V[i])
- sum_yv)

        V[i] = V_prev[i] + accel * (V_new - V_prev[i])

```

```

rel_err = np.max(np.abs((V - V_prev) / V))
rel_errs.append(rel_err)
history.append(V.copy())

converged = rel_err < tol
label = f"{iteration + 1}" + (" [done]" if converged else "") # final
iteration reached
print(f"\t{label:^{iter_w}}", end='')
for v in V:
    print(f"{abs(v):^11.6f}{np.degrees(np.angle(v)):^11.4f}", end='')
print(f"{rel_err:^{err_w}.2e}")

if converged:
    print(f"\t{sep}")
    print(f"\t Converged at iteration {iteration + 1} (e% =
{rel_err:.2e})")
    break
else:
    print(f"\t{sep}")
    print(f"\t Did not converge within {max_iter} iterations.")

print(f"\t{sep}\n")
return V, history, rel_errs

```

Line flow calculator function cell

Python

Power flow calculation through each line

```

def compute_line_flows(V, impedances):
    """
    Calculates complex power flowing through each transmission line.
    s_ij = V_i * conj( (V_i - V_j) / z_ij )
    Note: positive s_ij means power flows from bus i to bus j.
    """
    flows = {}
    for (i, j), z in impedances.items():
        ii, jj = i - 1, j - 1
        y_line = 1.0 / z
        I_ij = (V[ii] - V[jj]) * y_line # current from i to j
        S_ij = V[ii] * np.conj(I_ij) # apparent power from i
        S_ji = V[jj] * np.conj(-I_ij) # apparent power from j

```

```

    loss    = S_ij + S_ji                # total line loss
    flows[(i, j)] = {
        "S_ij_MW":    round(S_ij.real * 100, 4),
        "S_ij_MVAr":  round(S_ij.imag * 100, 4),
        "S_ji_MW":    round(S_ji.real * 100, 4),
        "S_ji_MVAr":  round(S_ji.imag * 100, 4),
        "loss_MW":     round(loss.real * 100, 4),
        "loss_MVAr":   round(loss.imag * 100, 4),
    }
    return flows

```

Slack bus power cell

Python

Slack-bus injection power calculation

```

def compute_slack_injection(V, Y):
    """
    Back-calculates the power injected by the slack bus (bus 1)
    using the converged voltages and the y-bus matrix.
    s_slack = V_1 * conj( sum_j Y_1j * V_j )
    """
    I_slack = Y[0] @ V
    S_slack = V[0] * np.conj(I_slack)
    return S_slack

```

Results terminal output function cell

Python

Results printing

```

def print_results(V, Y, bus_power, flows, rel_errs, bus_types=None):
    """
    Prints a formatted summary of the power flow solution.
    Note: defaults to the 4x4 example when other information not given.
    """

    n = len(V)

    # default bus-type map: reproduce the original 4-bus behaviour
    if bus_types is None:
        bus_types = {1: 'slack', 2: 'pv'}

```

```

        for b in range(3, n + 1):
            bus_types[b] = 'pq'

type_label = {'slack': 'slack', 'pv': 'PV (Gen.)', 'pq': 'PQ (Load)'}

print("\tVoltage estimation results")

print("\n\tBus voltages (converged solution):\n")
print(f"\t{'bus':<6} {'|V| (pu)':<12} {'angle (°)':<14} {'type':<10}")
print("\t" + "-" * 44)

for i, v in enumerate(V):
    bus = i + 1
    mag = abs(v)
    ang = np.degrees(np.angle(v))
    btype = type_label[bus_types[bus]]
    print(f"\t{'bus':<6} {'mag':<12.6f} {'ang':<14.6f} {'btype':<10}")

S_slack = compute_slack_injection(V, Y)
print(f"\n\tSlack bus injection:")
print(f"\tP = {S_slack.real * 100:.4f} MW")
print(f"\tQ = {S_slack.imag * 100:.4f} MVar")

print("\n\tLine power flows (base = 100 MVA):\n")
print(f"\t{'line':<10} {'P_ij(MW)':<12} {'Q_ij(MVar)':<14} {'loss_MW':<12}")
print("\t" + "-" * 50)
for (i, j), f in flows.items():
    print(f"\t{i}\u2192{j}<7} {f['S_ij_MW']:<12} {f['S_ij_MVar']:<14} {f['loss_MW']:<12}")

total_loss = sum(f["loss_MW"] for f in flows.values())
print(f"\n\tTotal real power losses: {total_loss:.4f} MW")
print(f"\tIterations to convergence: {len(rel_errs)}")

```

Results plotting function cell

```

Python
# Results plotting

```

```

def plot_results(V, history, rel_errs, flows, Y, impedances=None,
bus_types=None):
    """
    Generates a four-panel figure:
    (a) convergence curve - error vs iteration
    (b) voltage profile - magnitude at each bus
    (c) voltage phasors - polar representation
    (d) one-line diagram - microgrid topology with line flows

    impedances: optional dict {(from, to): Z} used for the one-line diagram.
                Defaults to the module-level line_impedances when omitted.
    bus_types: optional dict {bus_number: 'slack'|'pv'|'pq'}.
                Defaults to the original 4-bus example layout when omitted.
    """
    import math

    n = len(V)

    # fall back to module-level defaults so the original 4-bus call still works
    if impedances is None:
        impedances = line_impedances
    if bus_types is None:
        bus_types = {1: 'slack', 2: 'pv'}
        for b in range(3, n + 1):
            bus_types[b] = 'pq'

    color_palette = ["#6700db", "#21db12", "#94a1ed", "#dc12a0", "#ff8800",
"#00bcd4"]
    colors = color_palette[:n]

    type_short = {'slack': 'Slack', 'pv': 'PV Gen', 'pq': 'PQ Load'}
    bus_labels = [f"Bus {b}\n({type_short[bus_types[b]})" for b in range(1, n
+ 1)]

    fig = plt.figure(figsize=(16, 12))
    fig.patch.set_facecolor("#ffffff")
    gs = gridspec.GridSpec(2, 2, figure=fig, hspace=0.38, wspace=0.32)

    # panel a: convergence curve
    ax1 = fig.add_subplot(gs[0, 0])
    ax1.semilogy(range(1, len(rel_errs) + 1), rel_errs,
                color="#E91E63", linewidth=2, marker="o", markersize=4)
    ax1.axhline(TOLERANCE, color="gray", linestyle="--", linewidth=1,
label=f"tol = {TOLERANCE}")

```



```

ax1.set_xlabel("Iteration number", fontsize=10)
ax1.set_ylabel("Max voltage error (pu)", fontsize=10)
ax1.set_title("(a) Gauss-Seidel convergence", fontsize=11,
fontweight="bold")
ax1.legend(fontsize=9)
ax1.grid(True, which="both", alpha=0.4)
ax1.set_facecolor("#ffffff")

# panel b: voltage magnitude profile
ax2 = fig.add_subplot(gs[0, 1])
magnitudes = [abs(v) for v in V]
bars = ax2.bar(range(1, n + 1), magnitudes, color=colors,
edgecolor="white",
                linewidth=1.5, width=0.5)
ax2.axhline(0.95, color="black", linestyle="--", linewidth=1.2, label="0.95
pu lower limit")
ax2.axhline(1.05, color="black", linestyle="--", linewidth=1.2, label="1.05
pu upper limit")
for bar, mag in zip(bars, magnitudes):
    ax2.text(bar.get_x() + bar.get_width() / 2, mag + 0.002,
            f"{mag:.4f}", ha="center", va="bottom", fontsize=9,
fontweight="bold")
ax2.set_xticks(range(1, n + 1))
ax2.set_xticklabels(bus_labels, fontsize=9)
ax2.set_ylabel("|V| (per unit)", fontsize=10)
ax2.set_title("(b) Bus voltage magnitudes", fontsize=11, fontweight="bold")
ax2.set_ylim(0.90, 1.10)
ax2.legend(fontsize=8)
ax2.grid(axis="y", alpha=0.4)
ax2.set_facecolor("#ffffff")

# panel c: phasor diagram (polar)
ax3 = fig.add_subplot(gs[1, 0], projection="polar")
for v, col, lbl in zip(V, colors, bus_labels):
    angle = np.angle(v)
    mag = abs(v)
    ax3.annotate("", xy=(angle, mag), xytext=(0, 0),
                arrowprops=dict(arrowstyle="->", color=col, lw=2.0))
    ax3.plot(angle, mag, "o", color=col, markersize=8,
label=lbl.replace("\n", " "))
ax3.set_title("(c) Voltage phasors", fontsize=11, fontweight="bold",
pad=18)
ax3.legend(loc="upper right", bbox_to_anchor=(1.35, 1.15), fontsize=8)
ax3.set_facecolor("#ffffff")

```

```

# panel d: one-line diagram
ax4 = fig.add_subplot(gs[1, 1])
ax4.set_facecolor("#ffffff")
ax4.set_xlim(0, 10)
ax4.set_ylim(0, 10)
ax4.axis("off")
ax4.set_title("(d) Microgrid one-line diagram", fontsize=11,
fontweight="bold")

# distribute bus positions evenly around a circle
cx, cy, r = 5, 5, 3.5
pos = {}
for b in range(1, n + 1):
    theta = math.radians(90 - (b - 1) * 360 / n)
    pos[b] = (cx + r * math.cos(theta), cy + r * math.sin(theta))

# draw lines with flow annotations
for (i, j) in impedances.keys():
    x1, y1 = pos[i]
    x2, y2 = pos[j]
    ax4.plot([x1, x2], [y1, y2], "k-", linewidth=2, zorder=1)
    mx, my = (x1 + x2) / 2, (y1 + y2) / 2
    f = flows[(i, j)]
    ax4.text(mx, my + 0.35,
             f"P={f['S_ij_MW']:.2f}MW",
             ha="center", fontsize=7.5, color="#555555",
             bbox=dict(boxstyle="round,pad=0.2", fc="white", ec="none"))

# draw bus nodes
for bus, (x, y) in pos.items():
    ax4.scatter(x, y, s=600, color=colors[bus - 1], zorder=3,
edgecolors="white", linewidths=2)
    mag = abs(V[bus - 1])
    ang = np.degrees(np.angle(V[bus - 1]))
    ax4.text(x, y + 0.75,
             f"Bus {bus}\n|V|={mag:.4f} pu\n\u03b2={ang:.2f}\u00b0",
             ha="center", fontsize=8, fontweight="bold",
             bbox=dict(boxstyle="round,pad=0.3", fc=colors[bus - 1] + "33",
ec="none"))

plt.suptitle(
    f"Voltage estimation in a {n}-bus microgrid\n"
    "using the Gauss-Seidel iterative method",

```

```

        fontsize=12, fontweight="bold", y=0.99, color="#000000"
    )

plt.show()

```

Main Cell 1 (4x4 Output)

```

Python
# Main Function
if __name__ == "__main__":
    print("\tGauss-Seidel Power Flow on a Four-Bus Microgrid")
    print("\tNUMMETS-E32/EQ2 | Group 7\n")

    # build admittance matrix
    Y = build_ybus(line_impedances, n_buses=4)
    print("\tY-Bus Matrix:")
    for row in Y:
        print("\t", ["{:+.4f}".format(v) for v in row])
    print("\n")

    # run gauss-seidel iterations
    V_sol, history, rel_errs = gauss_seidel_power_flow(Y, bus_power,
bus_voltage_spec)

    # compute line flows
    flows = compute_line_flows(V_sol, line_impedances)

    # print summary
    print_results(V_sol, Y, bus_power, flows, rel_errs)

    # generate plots
    plot_results(V_sol, history, rel_errs, flows, Y)

```

User input helper functions cell

```

Python
# User-Input: helper utilities
"""
Shared helper for parsing user-typed complex numbers.
"""
import math
def _parse_complex(s):

```

```

"""
Parses a user-supplied complex number.
Accepts plain Python complex literals such as 0.02+0.06j or -0.55-0.18j.
"""

s = s.strip()
try:
    return complex(s)
except ValueError:
    raise ValueError(f"Cannot parse '{s}' as a complex number.")

def _prompt_voltage(prompt, default):
    """
    Prompt for a complex voltage with a typed default.
    Returns the default unchanged if the user presses Enter.
    """
    raw = input(prompt).strip()
    if not raw:
        return default
    try:
        return _parse_complex(raw)
    except ValueError as e:
        print(f"  [!] {e} - using default {default}")
        return default

print("Gauss-Seidel Power Flow, Interactive Configuration")
while True:
    raw = input("\nHow many buses? (4-6): ").strip()
    try:
        n_buses = int(raw)
        if 4 <= n_buses <= 6:
            break
        print("\t[!] Enter a value between 4 and 6.")
    except ValueError:
        print("\t[!] Please enter a whole number.")

print(f"\nConfiguring a {n_buses}-bus microgrid.")

```

Bus-type input cell

```

Python
# User-Input: bus-type assignment

"""

```

Assigns a type to every bus.
 Bus 1 is always the slack (reference) bus.
 All other buses are either PV (generator, fixed $|V|$) or PQ (load, fixed $P+jQ$).
 """

```
print("\tBus 1 is always the SLACK bus.")
bus_types_map = {1: 'slack'}

for bus in range(2, n_buses + 1):
    while True:
        raw = input(f"\tBus {bus} type: PV (generator) or PQ (load)? [pv/pq]:")
        raw = raw.strip().lower()
        if raw in ('pv', 'pq'):
            bus_types_map[bus] = raw
            break
        print("\t[!] Type 'pv' or 'pq'.")

# echo summary
print("\n\tBus-type summary:")
type_label = {'slack': 'slack', 'pv': 'PV (Gen.)', 'pq': 'PQ (Load)'}
for b in range(1, n_buses + 1):
    print(f"\t\tBus {b}: {type_label[bus_types_map[b]]}")
```

Line impedance input cell

Python

User-Input: line impedances

"""

Enter the impedance of every transmission line.
 Format: from_bus to_bus R+Xj; e.g. 1 2 0.02+0.06j (no spaces at R+Xj)
 Stops at a blank entry (ENTER).
 """

```
print("\nEnter line impedances (resistance + j*reactance, in per-unit).")
print("Format: from_bus to_bus R+Xj; e.g. 1 2 0.02+0.06j")
print("Press ENTER on a blank line when finished.\n")
```

```
user_impedances = {}
while True:
    raw = input("\tConnection (or blank to stop): ").strip()
    if not raw:
        if not user_impedances:
```

```

        print("\t[!] At least one connection is required.")
        continue
    break
parts = raw.split()
if len(parts) != 3:
    print("\t[!] Expected format:  from_bus  to_bus  R+Xj")
    continue
try:
    fb, tb = int(parts[0]), int(parts[1])
    z = _parse_complex(parts[2])
except (ValueError, IndexError) as e:
    print(f"\t[!] {e}")
    continue
if not (1 <= fb <= n_buses and 1 <= tb <= n_buses and fb != tb):
    print(f"\t[!] Bus numbers must be between 1 and {n_buses} and must differ.")
    continue
if z == 0:
    print("\t[!] Impedance cannot be zero.")
    continue
user_impedances[(fb, tb)] = z
print(f"\tStored: ({fb}, {tb})  Z = {z}")

print(f"\n  {len(user_impedances)} line(s) configured.")

```

Power injection input cell

```

Python
# User-Input: scheduled power injections

"""
Enter the scheduled complex-power injection at each non-slack bus.
  PV buses : active power P only (e.g.  0.40)
  PQ buses : P + jQ (e.g. -0.55-0.18j  for a load)
Bus 1 (slack) is skipped, its injection is back-calculated after the solution.
"""

print("\nEnter scheduled complex-power injections (per-unit).")
print("\tPV buses  : enter active power only  e.g.  0.40")
print("\tPQ buses  : enter P + jQ                e.g.  -0.55-0.18j  for a load")
print("Bus 1 (slack) is skipped; its injection is calculated automatically.\n")

```

```

user_bus_power = {1: (None, None)}    # slack bus: p and q determined by
solution

for bus in range(2, n_buses + 1):
    btype = bus_types_map[bus]
    while True:
        if btype == 'pv':
            raw = input(f"\tBus {bus} (PV): scheduled P [pu]: ").strip()
        else:
            raw = input(f"\tBus {bus} (PQ): scheduled P+Qj [pu]: ").strip()
        if not raw:
            print(f"\t[!] A value is required for bus {bus}.")
            continue
        try:
            s = _parse_complex(raw)
        except ValueError as e:
            print(f"\t[!] {e}")
            continue
        if btype == 'pv':
            # only P matters; Q is computed each iteration
            user_bus_power[bus] = (s.real, 0.0)
            print(f"\tStored: bus {bus} -> P = {s.real:.4f} pu (Q computed
each iteration)")
        else:
            user_bus_power[bus] = (s.real, s.imag)
            print(f"\tStored: bus {bus} -> P = {s.real:.4f} pu, Q =
{s.imag:.4f} pu")
        break

```

Voltage initial guess/specification cell

```

Python
# User-Input: initial voltage specifications

"""
Specify the initial voltage at each bus.
    Bus 1 (slack): fixed complex voltage (default 1.05+0j)
    PV buses: fixed magnitude only (default 1.02)
    PQ buses: flat-start complex guess (default 1.0+0j)
Defaults are set when user enters blank (ENTER).
"""

```

```

print("\nEnter voltage specifications (press ENTER to accept the default).")

user_bus_voltage_spec = {}

# bus 1: slack, full complex voltage
v1 = _prompt_voltage("\tBus 1 slack voltage    [default 1.05+0j]: ",
complex(1.05, 0.0))
user_bus_voltage_spec[1] = v1

for bus in range(2, n_buses + 1):
    btype = bus_types_map[bus]
    if btype == 'pv':
        # pv bus: magnitude only (float)
        raw = input(f"\tBus {bus} PV magnitude    [default 1.02]: ").strip()
        try:
            mag = float(raw) if raw else 1.02
        except ValueError:
            print("\t[!] Invalid - using default 1.02")
            mag = 1.02
        user_bus_voltage_spec[bus] = mag
    else:
        # pq bus: flat-start complex guess
        v = _prompt_voltage(
            f"\tBus {bus} PQ initial guess [default 1.0+0j]: ",
            complex(1.0, 0.0)
        )
        user_bus_voltage_spec[bus] = v

```

User input analysis and main cell

Python

User-Input: run the analysis

"""

Builds the Y-bus matrix, runs the Gauss-Seidel iterations, computes line flows, then prints results and plots; all using the same functions defined in Cells 1-6, now called with the user-supplied data.

"""

```
print("\n\tRunning Gauss-Seidel Power Flow with user-supplied data")
```

build admittance matrix


```

Y_user = build_ybus(user_impedances, n_buses=n_buses)
print("\n\tY-Bus Matrix:")
for row in Y_user:
    print("\t", ["{:+.4f}".format(v) for v in row])
print("\n")

# run gauss-seidel iterations
V_user, history_user, rel_errs_user = gauss_seidel_power_flow(
    Y_user, user_bus_power, user_bus_voltage_spec, bus_types=bus_types_map
)

# compute line flows
flows_user = compute_line_flows(V_user, user_impedances)

# print summary
print_results(V_user, Y_user, user_bus_power, flows_user, rel_errs_user,
              bus_types=bus_types_map)

# generate plots
plot_results(V_user, history_user, rel_errs_user, flows_user, Y_user,
             impedances=user_impedances, bus_types=bus_types_map)

```

Appendix B - Member Contribution

Cuartero, Gavin Joaquin - writing Introduction and Objective sections

Tafalla, Carl Andrei - Programming and implementation, writing Methodology and User Input-specific section, and final revisions to all other sections

Tonducan, Zach Nathan - writing Results and Discussion and Conclusion section